43 WEB CODER

POWER OF TEAMWORK

## Bài 6

# Spring boot và JPA

MODULE 4

# Mục tiêu



- Giải thích được mô hình hoạt động của Spring Boot
- Cài đặt và sử dụng được Spring Boot để phát triển ứng dụng
- Trình bày được JPA
- Trình bày được Entity
- Trinh bày được Entity Manager
- Trình bày được hoạt động của Spring Data JPA
- Triển khai được Spring Data JPA để thao tác cơ bản với CSDL



# Spring boot o**verview**

Spring làm cho việc phát triển một ứng dụng Java trở lên dễ dàng...

Nhưng Spring Boot còn có thể làm nó dễ dàng hơn!

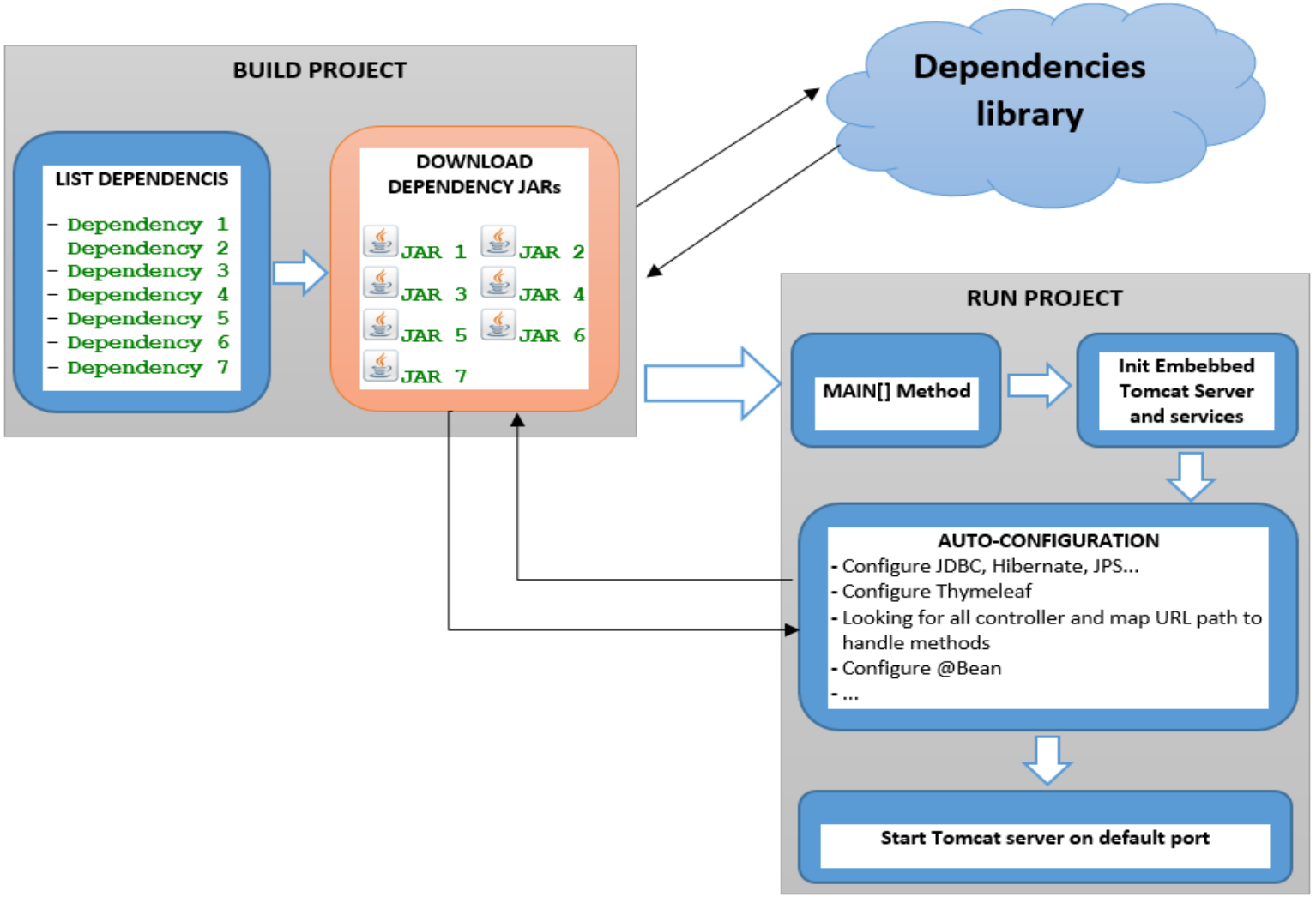
*- Craig Walls -*

# Overview



- Spring Boot là giải pháp cho việc cấu hình một project độc lập của Spring.
- Spring Boot giúp cho việc bắt đầu một dự án Spring trở lên dễ dàng và nhanh chóng.
- Spring Boot được thể hiện thông qua một loạt các quy tắc nhằm mục đích.
  - Cung cấp ra các ứng dụng đã được cấu hình và đóng gói đầy đủ có thể chạy độc lập.
  - Giảm thiểu tối đa sự phức tạp của việc cấu hình project cũng như quản lý các thư viện được cung cấp từ bên thứ 3 ngoài Spring.

# Các thức hoạt động



# Cách thức hoạt động



- Cũng như các dự án Spring, Spring Boot sử dụng Gradle, Maven... để khai báo, quản lý và build các thư viện.
- Các dự án Spring Boot bắt đầu khởi chạy từ hàm main[] nằm trong một class được xác định thông qua annotation @SpringBootApplication

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) { SpringApplication.run(Application.class, args); }

}
```

# Cách thức hoạt động



- Tomcat server được cấu hình nhúng sẵn trong một ứng dụng Spring Boot.
- Từ các thư viện dependencies đã khái báo, Spring Boọt tự động cấu hình các thành phần cần thiết cho ứng dụng.

# Đặc điểm(1)



- Spring Boot đưa ra 4 đặc điểm để phục vụ tối đa cho mục đích làm đơn và dễ dàng hơn trong việc phát triển dự án Spring:
  - Spring Boot starters
  - Auto-configuration
  - Command-line interface (CLI)
  - Actuator

# Spring Boot starters

- Cung cấp một single dependency (spring-boot-starter) chứa những denpencies có chung đặc điểm.

# Spring Boot starters



- Ví dụ:
  - Bắt đầu một dứng dụng RESTful Spring MVC, thao tác với dữ liệu từ database, kiểu dữ liệu là JSON và sử dụng Thymeleaf cho views, so sánh danh sách dependency giữa Spring MVC và Spring Boot.

| Spring MVC | Spring Boot |
|---|---|
| dependencies {<br>Compile("org.springframework:spring-web:4.0.6.RELEASE")<br>Compile("org.springframework:spring-webmvc:4.0.6.RELEASE")<br>Compile("com.fasterxml.jackson.core:jackson-databind:2.2.2")<br>Compile("org.springframework:spring-jdbc:4.0.6.RELEASE")<br>Compile("org.springframework:spring-tx:4.0.6.RELEASE")<br>Compile("org.thymeleaf:thymeleaf-spring4:2.1.2.RELEASE")<br>} | dependencies {<br>Compile("org.springframework.boot:spring-boot-starter-web:1.1.4.RELEASE")<br>Compile("org.springframework.boot:spring-boot-starter-jdbc:1.1.4.RELEASE")<br>Compile ("org.springframework.boot:spring-boot-starter-thymeleaf")<br>} |

# Spring Boot starters



- Ưu điểm:
  - Rút ngắn danh sách dependency cần khai báo cho 1 dự án Spring.
  - Giảm bớt công việc phải làm trong quá trình khai báo và quản lý các dependencies.
  - Đảm bảo tất cả dependencies hoạt động ổn định cùng nhau.
  - Tạo sự rõ rang, dễ đoán được tác dụng từ việc đặt tên các dependencies.

# Auto-configuration



- Đặc điểm auto-configuration của Spring Boot sẽ tự động cấu hình dự án dựa trên các dependencies đã được thêm vào Gradle/Maven.
- Khởi động auto-configuration thông qua antonation @EnableAutoconfiguration hoặc @SpringBootApplication.
- Ví dụ:
  - Để sử dụng Thymeleaf cho việc xử lý các view trong một dự án:
    - Với SpringMVC cần cấu hình tối thiểu 3 đối tượng *ThymeleafView-Resolver*, *SpringTemplateEngine* và *TemplateResolver.*
    - Với Spring Boot việc duy nhất cần làm là khai báo 1 dependency của Thymeleaf trong Gradle/ Maven. Spring Boot sẽ tự động cấu hình *ThymeleafView-Resolver*, *SpringTemplateEngine* và *TemplateResolver* cho dự án.

# Auto-configuration



- Ưu điểm:
  - Giảm thiểu rất nhiều khối lượng đội tượng cần cấu hình khi bắt đầu một dự án Spring.
  - Làm đơn giản cấu trúc một dự án Spring giúp người phát triển tập trung nhiều hơn vào phát triển logic nghiệp vụ.
  - Giúp việc chuyển đổi giữa các công nghệ, thư viện dễ dàng hơn.



# Triển khai SpringBoot

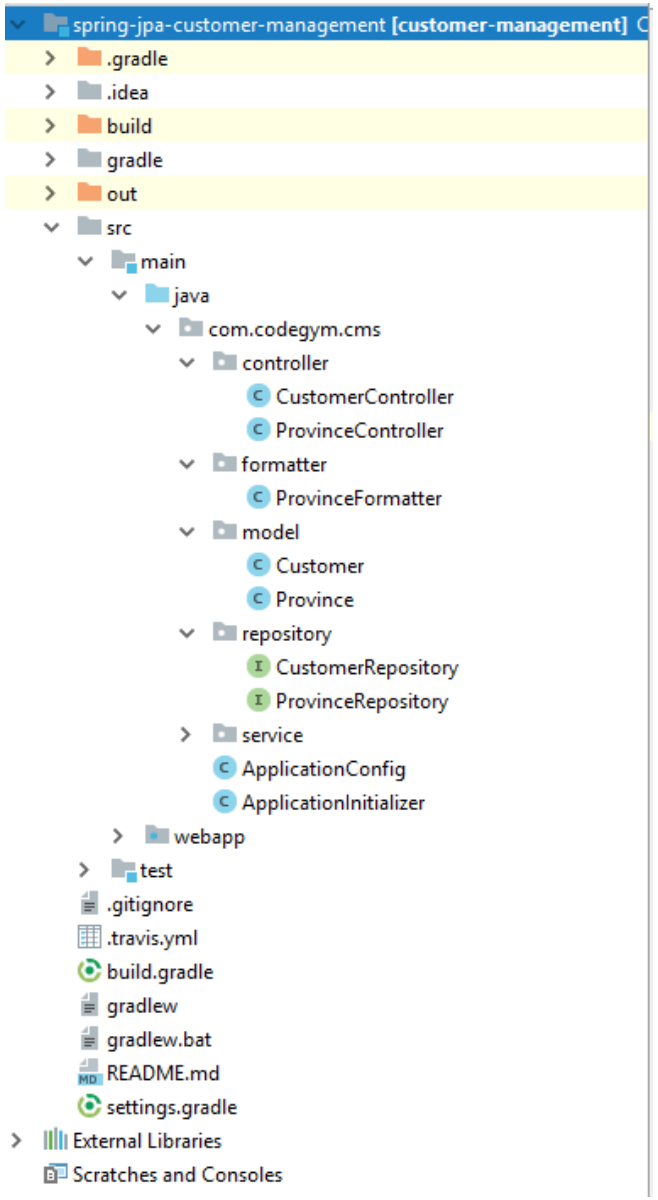
# Triển khai Spring Boot (1)



- Hiện nay các công cụ lập trình như Intellij, Eclipse...đều hỗ trợ người sử dụng tạo 1 Spring Boot project mang đầy đủ các thuộc tính của Spring Boot hết sức đơn giản.
- Để làm rõ sự khác biệt giữa 1 ứng dụng Spring MVC thông thường, và Spring Boot, tài liệu này sẽ triển khai 1 ứng dụng từ Spring MVC sang Spring Boot.
- Ứng dụng Spring MVC được sử dụng là bài tập đã làm Chương 4.

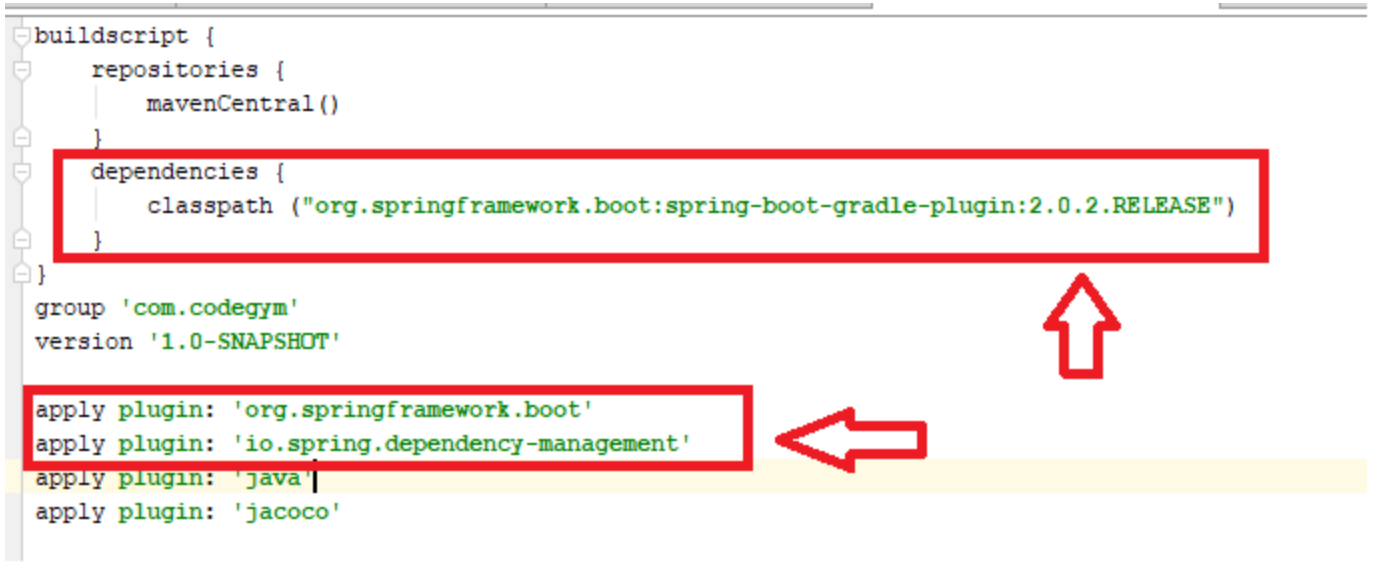
# Triển khai Spring Boot (2)

- Cấu trúc dự án Spring MVC



# Triển khai Spring Boot (3)

- Phân tích:
  - Spring Boot với đặc điểm "Spring Boot starters" sẽ sử dụng gói dependency mới để thay thế cho đa số các dependency cũ của Spring MVC nên chắc chắn cấu trúc file Gradle sẽ có thay đổi.
    - Thêm thư viện Spring-boot-gradle-plugin cho dự án, version của gradle plugin sẽ quyết định version của các dependencies khai báo bên dưới.



# Triển khai Spring Boot (4)



- Thay thế các dependencies của Spring MVC bằng các dependencies trong gói Spring-boot-starter.

```
dependencies {
    //--------------Spring MVC dependencies -------------------------------------
//    compile group: 'org.thymeleaf', name: 'thymeleaf-spring4', version: '3.0.9.RELEASE'
//    compile group: 'javax.servlet', name: 'javax.servlet-api', version: '3.1.0'
//    compile group: 'org.springframework', name: 'spring-core', version: '4.3.17.RELEASE'
//    compile group: 'org.springframework', name: 'spring-context', version: '4.3.17.RELEASE'
//    compile group: 'org.springframework', name: 'spring-beans', version: '4.3.17.RELEASE'
//    compile group: 'org.springframework', name: 'spring-web', version: '4.3.17.RELEASE'
//    compile group: 'org.springframework', name: 'spring-orm', version: '4.3.17.RELEASE'
//    compile group: 'org.springframework', name: 'spring-webmvc', version: '4.3.17.RELEASE'
//    compile group: 'org.springframework.data', name: 'spring-data-jpa', version: '1.11.12.RELEASE'
    //--------------Spring MVC dependencies -------------------------------------

    //--------------Spring Boot Starter dependencies -----------------------------
    compile ("org.springframework.boot:spring-boot-starter-parent")
    compile ("org.springframework.boot:spring-boot-starter-web")
    compile ("org.springframework.boot:spring-boot-starter-data-jpa")
    compile ("org.springframework.boot:spring-boot-starter-thymeleaf")
    //--------------Spring Boot Starter dependencies -----------------------------
```

# Triển khai Spring Boot (5)



- Phân tích:
  - Với đặc điểm "Auto-configuration" Spring Boot sẽ dựa trên danh sách dependency để tự động cấu hình các thành phần cần thiết cho ứng dụng.
  - Những cấu hình thủ công bên dưới đều có thể bỏ qua:
    - ApplicationInitializer
    - ApplicationContext
    - Thymeleaf configuration (*templateResolver, templateEngine, ThymeleafViewResolver*)
    - JPA configuration
    - Data sources, transactionManager...

# Triển khai Spring Boot (6)



- Xóa bỏ 2 class ApplicationInitizlizer và ApplicationConfig, thay vào đó là 1 class Application và hàm Main[] như sau:

```
package com.codegym.cms;

import com.codegym.cms.service.CustomerService;
import com.codegym.cms.service.ProvinceService;
import com.codegym.cms.service.impl.CustomerServiecImpl;
import com.codegym.cms.service.impl.ProvinceServiceImpl;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class Application {

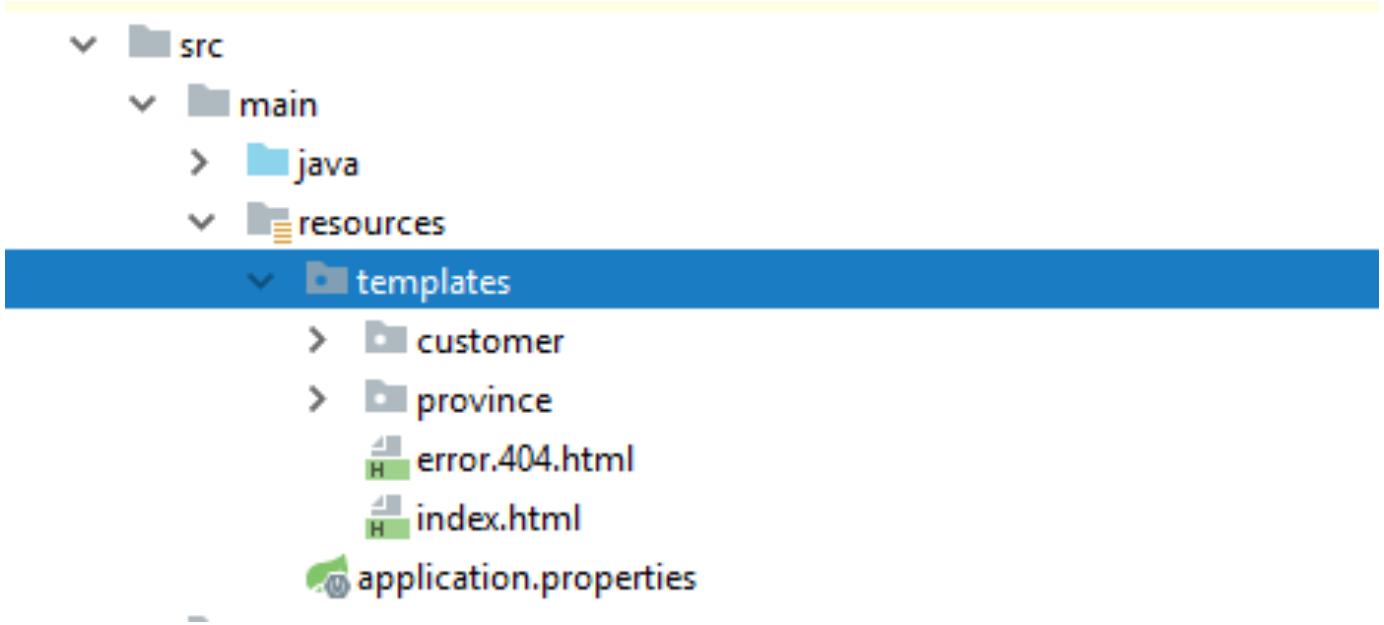
    @Bean
    public CustomerService customerService(){ return new CustomerServiecImpl(); }
    @Bean
    public ProvinceService provinceService(){ return new ProvinceServiceImpl(); }

    public static void main(String[] args) { SpringApplication.run(Application.class, args); }

}
```

# Triển khai Spring Boot (7)

- Phân tích:
  - Spring Boot quản lý các views tại thư mực *src/main/resources/template.*
  - Move tất cả các view trong thư mục src/main/webapp/WEB-INF/views sang thư mục template ở trên.



# Triển khai Spring Boot (8)

- Phân tích:
  - Spring Boot tự động cấu hình kết nối đến database nhưng vẫn cần cung cấp các thông tin liên quan đến driver, url, username, password.
  - Các thông tin nêu trên được lưu trong file *src/main/resources/*application.properties

```
#-------- MySQL database -------------------
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/cms?useSSL=false
spring.datasource.username=root
spring.datasource.password=M0209t2304

#------------Hibernate-----------------------
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=none
spring.jpa.properties.hibernate.hbm2ddl.auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

# Triển khai Spring Boot (9)



- Chạy ứng dụng Spring Boot bằng cách chạy hàm Main[].



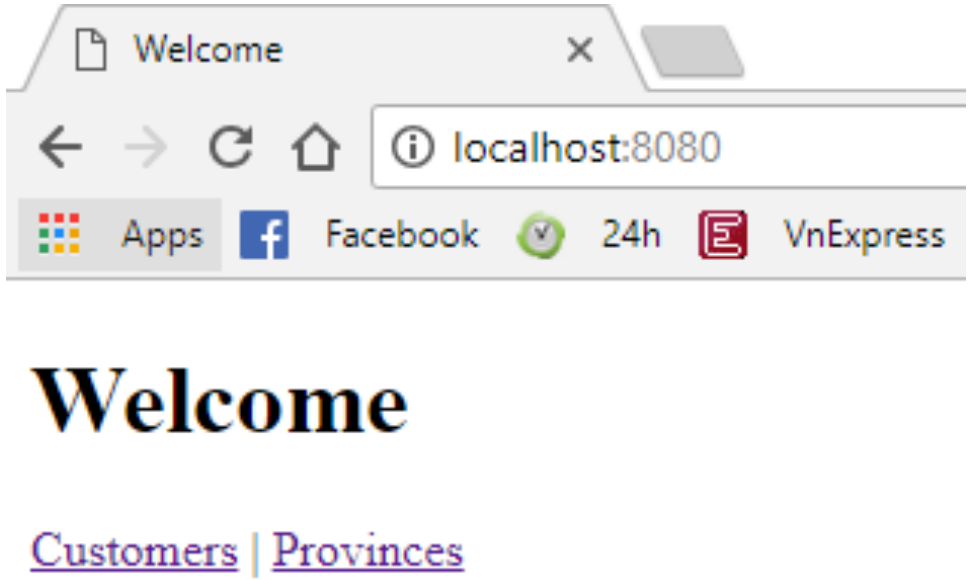
```
"C:\Program Files\Java\jdk1.8.0_162\bin\java.exe" ...

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.0.2.RELEASE)

2018-06-10 20:18:50.873  INFO 1676 --- [           main] com.codegym.cms.Application              : Starting Application on Dell-M4800 with PID
2018-06-10 20:18:50.876  INFO 1676 --- [           main] com.codegym.cms.Application              : No active profile set, falling back to defa
2018-06-10 20:18:50.926  INFO 1676 --- [           main] ConfigServletWebServerApplicationContext : Refreshing org.springframework.boot.web.ser
2018-06-10 20:18:51.655  INFO 1676 --- [           main] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.transaction.annot
2018-06-10 20:18:51.855  INFO 1676 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http
2018-06-10 20:18:51.872  INFO 1676 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2018-06-10 20:18:51.873  INFO 1676 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet Engine: Apache Tomcat/8.5.
2018-06-10 20:18:51.876  INFO 1676 --- [ost-startStop-1] o.a.catalina.core.AprLifecycleListener   : The APR based Apache Tomcat Native library
2018-06-10 20:18:51.984  INFO 1676 --- [ost-startStop-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicatior
2018-06-10 20:18:51.985  INFO 1676 --- [ost-startStop-1] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization
2018-06-10 20:18:52.086  INFO 1676 --- [ost-startStop-1] o.s.b.w.servlet.ServletRegistrationBean  : Servlet dispatcherServlet mapped to [/]
2018-06-10 20:18:52.089  INFO 1676 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'characterEncodingFilter' t
2018-06-10 20:18:52.090  INFO 1676 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'hiddenHttpMethodFilter' to
2018-06-10 20:18:52.090  INFO 1676 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'httpPutFormContentFilter'
2018-06-10 20:18:52.090  INFO 1676 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean   : Mapping filter: 'requestContextFilter' to:
2018-06-10 20:18:52.195  INFO 1676 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2018-06-10 20:18:52.270  INFO 1676 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2018-06-10 20:18:52.300  INFO 1676 --- [           main] j.LocalContainerEntityManagerFactoryBean : Building JPA container EntityManagerFactory
2018-06-10 20:18:52.310  INFO 1676 --- [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [
```

# Triển khai Spring Boot (9)

- Mở trình duyệt truy cấp http://localhost:8080/



# Triển khai Spring Boot (10)



- Triển khai Actuator cho 1 dự án Spring Boot
  - Thêm dependency cho Actuator.

```
compile ("org.springframework.boot:spring-boot-starter-actuator")
```

  - Thêm tham số cấu hình cho Actuator trong application.properties

```
server.port=8080
management.server.port=8090
management.endpoints.web.exposure.include=*
management.endpoint.shutdown.enabled=true
```

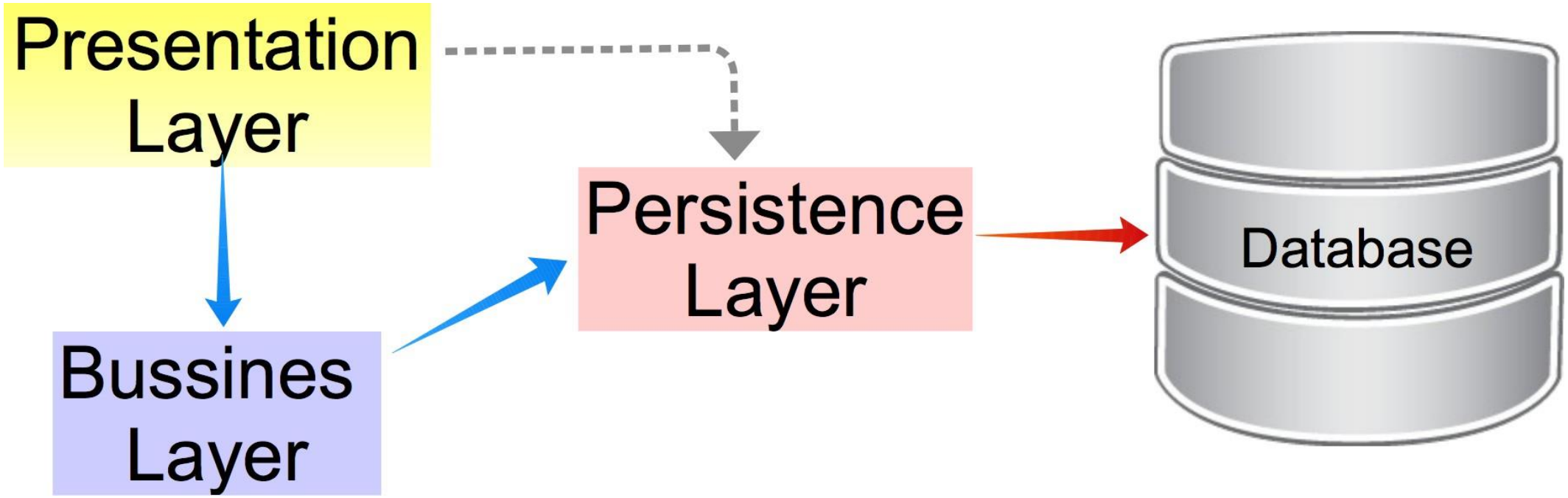
  - Chạy ứng dụng, truy cập http://localhost:8090/actuator để xem kết quả.
  - Có thể tìm hiểu thêm các endpoint khác như **/actuator/health, /actuator/beans, /actuator/env …**



# Thảo luận

JPA – Java Persisntence API

# Persistence Layer



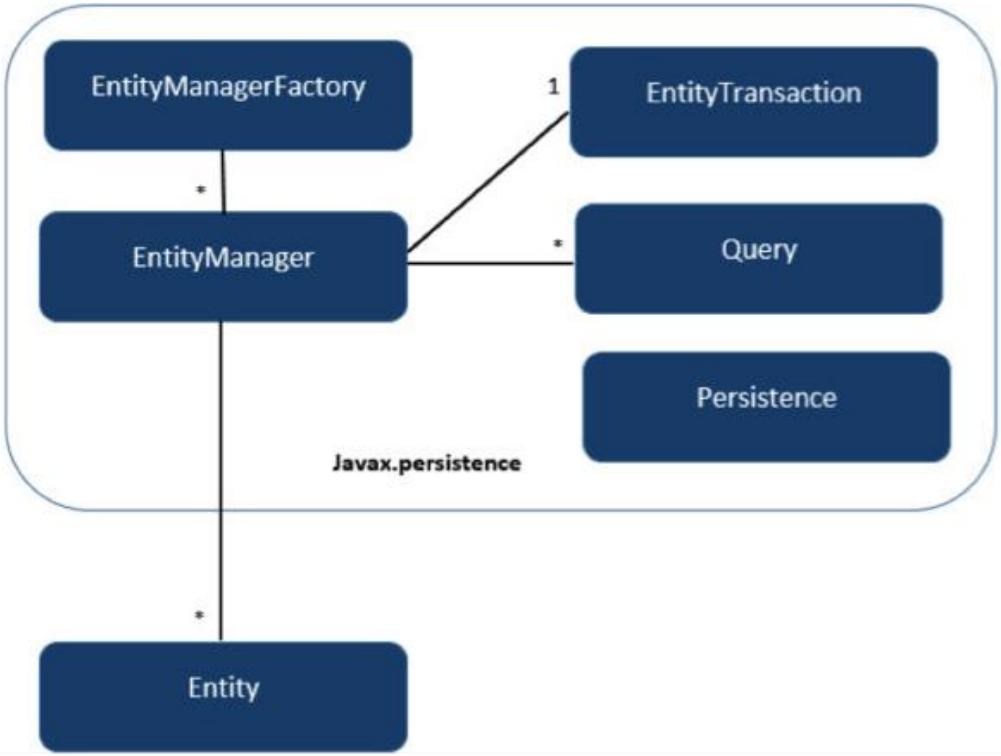
# Entity



- Entity là đối tượng đại diện cho dữ liệu ở trong ứng dụng
- Entity thường là POJO (Plain Old Java Object)
- Entity sẽ được ánh xạ (mapping) tới một bảng trong CSDL
- Trong JPA, một entity cần tuân thủ:
  - Được gắn với annotation javax.persistence.Entity
  - Có một constructor là public và không có tham số
  - Không được khai báo final
  - Các biến đối tượng cần được khai báo là private, protected hoặc ở mức package-private

# Persistence Context & Entity Manager

- Persistence Context là tập các thể hiện của entity được quản lý, tồn tại trong một kho dữ liệu
- Interface EntityManager:
  - Khai báo các phương thức để tương tác với persistence context
  - Tạo hoặc xoá các thể hiện của entity
  - Tìm kiếm entity theo khoá chính
  - Thực thi các câu lệnh truy vấn lên entity



# Quản lý Entity



- Container-managed Entity Manager:

```
@PersistenceContext
EntityManager em;
```

- Application-managed Entity Manager:

```
@PersistenceUnit
EntityManagerFactory
emf;

EntityManager em = emf.createEntityManager();
```

# Tìm một Entity



- Ví dụ:

```
@PersistenceContext
EntityManager em;
public void enterOrder(int custID, Order
  newOrder) { Customer cust =
  em.find(Customer.class, custID);
  cust.getOrders().add(newOrder);
  newOrder.setCustomer(cust);
}
```

# Lưu trữ Entity



- Ví dụ:

```
@PersistenceContext
EntityManager em;
...
public LineItem createLineItem(Order order, Product product, int
   quantity) { LineItem li = new LineItem(order, product,
   quantity); order.getLineItems().add(li);
   em.persist(li);
   return li;
}
```

# Xoá Entity



- Ví dụ:

```
public void removeOrder(Integer orderId) {
  try {
    Order order = em.find(Order.class, orderId);
    em.remove(order);
  }...
```

# Câu lệnh truy vấn động



- Phương thức createQuery() của lớp EntityManager giúp tạo các câu truy vấn động (dynamic query)
- Ví dụ:

```
public List findWithName(String name) {
  return em.createQuery(
      "SELECT c FROM Customer c WHERE c.name LIKE
      :custName")
      .setParameter("custName", name)
      .setMaxResults(10)
      .getResultList();
}
```

# Named Query



- Phương thức createNamedQuery() của lớp EntityManager giúp tạo các câu truy vấn tĩnh (static query)
- Ví dụ, khai báo Named Query:

```
@NamedQuery(
    name="findAllCustomersWit
    hName",
    query="SELECT c FROM Customer c WHERE c.name LIKE :custName"
)
```

- Sử dụng Named Query

```
@PersistenceContext
public EntityManager em;
    ...
    customers = em.createNamedQuery("findAllCustomersWithName")
    .setParameter("custName", "Smith")
    .getResultList();
```

# Named Parameter



- Tên của các tham số bắt đầu bằng dấu (:)
- Ví dụ:

```
public List findWithName(String name) {
  return em.createQuery(
       "SELECT c FROM Customer c WHERE c.name LIKE :custName")
       .setParameter("custName", name)
       .getResultList();
}
```

- Sử dụng phương thức setParameter() để truyền giá trị

# Positional Parameter



- Vị trí của các tham số bắt đầu bằng dấu (?)
- Ví dụ:

```
public List findWithName(String name) {
  return em.createQuery(
      "SELECT c FROM Customer c WHERE c.name LIKE ?1")
      .setParameter(1, name)
      .getResultList();
}
```



# Demo

JPA – Java Persisntence API

# Quản lý Transaction



```
@PersistenceContext
  EntityManagerFactory
  emf; EntityManager
  em; @Resource
  UserTransaction utx;
  ......
        em = emf.createEntityManager();
        try {
              utx.begin();
              em.persist(SomeEntit
              y);
              em.merge(AnotherEn
              tity);
              em.remove(ThirdEntit
              y); utx.commit();
        } catch (Exception e) {
              utx.rollback();
        }
```



# Thảo luận

Spring Data JPA

# Spring Data JPA



Cải tiến JPA tiêu chuẩn

Đơn giản hoá tầng truy xuất dữ liệu

Tự tạo repository

Truy vấn DSL

Ghi log, phân trang

Có thể tuỳ biến nếu cần thiết

# Lựa chọn Tầng truy xuất dữ liệu



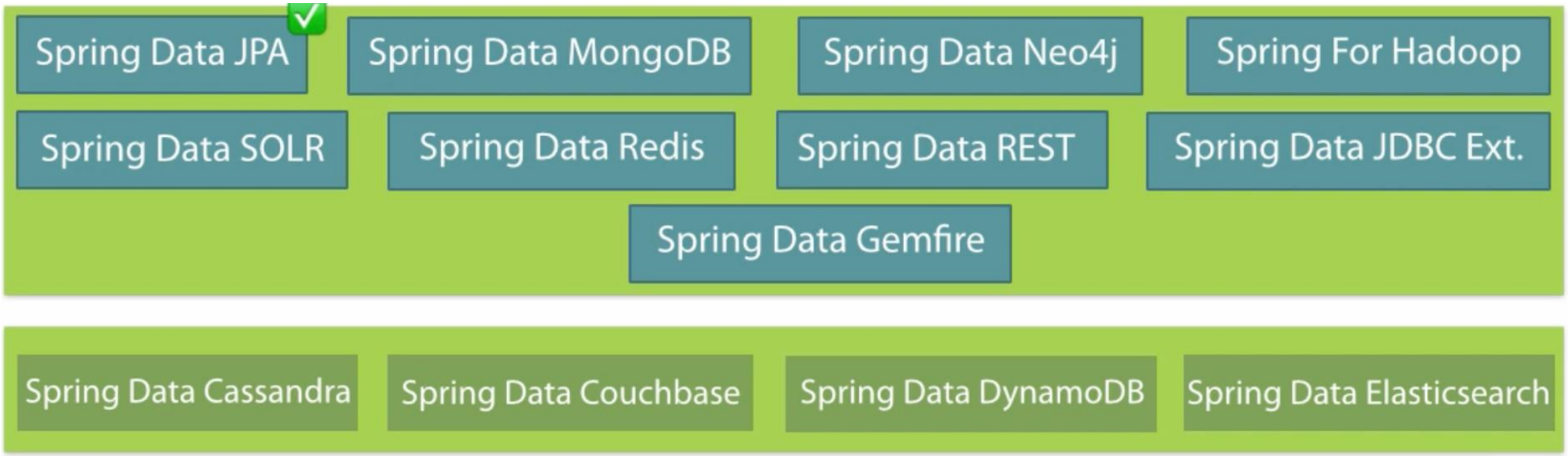
| JDBC<br>Spring JDBC | JEE 7 Batch<br>Spring Batch Hadoop | ORM<br>JPA/Hibernate<br>Spring Data JPA | NoSQL<br>MongoDB<br>Spring Data Mongo |
|---|---|---|---|
| • Đơn giản<br>• Thuần SQL | • Rất nhiều câu lệnh ghi SQL được thực hiện | • Dễ truy xuất các mối quan hệ | • Nhóm các dữ liệu có quan hệ với nhau |

# Spring Data



Spring Data Commons

- Repository
- Cross-Store persistency
- Dynamic query generation



# Cấu hình Spring Data JPA



build.gradle

```
dependencies {
  compile group: 'org.springframework.data', name: 'spring-data-jpa', version:
  '2.0.7.RELEASE'
  compile group: 'jstl', name: 'jstl', version: '1.2'
  compile group: 'mysql', name: 'mysql-connector-java', version: '8.0.11'
  testCompile group: 'org.junit.jupiter', name: 'junit-jupiter-engine', version:
  '5.2.0'
}
```

# Cấu hình DataSource



- Có thể cấu hình thông qua XML hoặc Annotation:

```
@Bean
public DataSource dataSource(){
   DriverManagerDataSource dataSource = new
   DriverManagerDataSource();
   dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
   dataSource.setUrl("jdbc:mysql://localhost:3306/phone_store");
   dataSource.setUsername( "root" );
   dataSource.setPassword( "123456" );
   return dataSource;
}
```

# Cấu hình Entity Manager



```
@Bean
@Qualifier(value = "entityManager")
public EntityManager entityManager(EntityManagerFactory entityManagerFactory) {
  return entityManagerFactory.createEntityManager();
}

@Bean
public LocalContainerEntityManagerFactoryBean entityManagerFactory()
  { LocalContainerEntityManagerFactoryBean em
       = new LocalContainerEntityManagerFactoryBean();
  em.setDataSource(dataSource());
  em.setPackagesToScan(new String[] { "com.codegym.phonestore.model" });

  JpaVendorAdapter vendorAdapter = new
  HibernateJpaVendorAdapter();
  em.setJpaVendorAdapter(vendorAdapter);
  em.setJpaProperties(additionalProperties());
  return em;
}
```

# Cấu hình Model



```
@Entity
@Table(name =
"phones") public
class Phone {
   @Id
   @GeneratedValue(strategy = GenerationType.AUTO)
   private Long id;

   @Column(nullable =
   false) private String
   name;

   @ManyToOne(targetEntity = Manufacture.class, cascade = {CascadeType.PERSIST,
   CascadeType.REMOVE})
   private Manufacture manufacture;

   //Constructors
   //Getters/Setters
}
```

# Truy xuất dữ liệu qua Entity Manager



```
@Transactional
public class CountryRepositoryImpl implements CountryRepository {

   @PersistenceContext
   EntityManager em;

   @Override
   public List<Country> findAll() {
     TypedQuery<Country> query = em.createQuery("select c from Country c",
     Country.class);
     return query.getResultList();
   }

   @Override
   public Country findById(Long id) {
     return em.find(Country.class, id);
   }
  .......
}
```

# Các thao tác cơ bản



```
public interface Repository
   <T> { List<T> findAll();

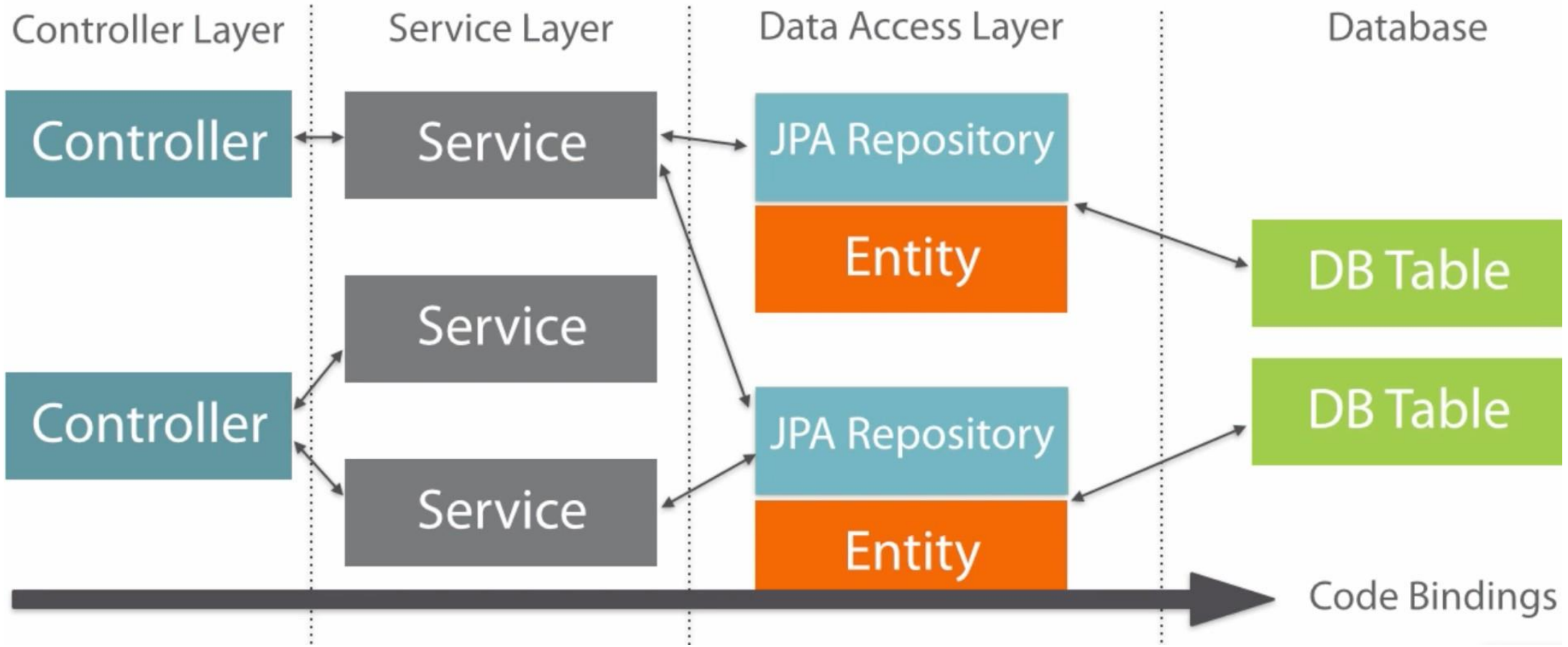
   T findById(Long

   id); T save(T

   model);

   void remove(Long id);
}
```

# Kiến trúc Repository





# Demo

Spring Data JPA

# Tổng kết



- JPA là đặc tả của Java dành cho các thao tác với dữ liệu
- Entity là các đối tượng đại diện cho dữ liệu
- Entity Manager là đối tượng quản lý các entity
- Spring Data JPA là framework của Spring hỗ trợ JPA, giúp cho việc triển khai JPA trở nên thuận tiện hơn